Epistemic Security: Enforceable Admission and the Limits of Counterfactual Defense

Anonymous Author(s)
Submission to the IEEE Symposium on Security and Privacy

Abstract—Agentic AI systems act on beliefs derived from adversarially writable content. We formalize decision-level epistemic integrity (DEI) as five invariants and prove that the first four are execution-monitor-enforceable safety properties, while the fifth, counterfactual dependence, is a 2-safety hyperproperty that no single-trace monitor can evaluate. We instantiate the enforceable fragment in CZTA, a zero-trust MCP gateway, as the physical realization of Algorithm 1 and Proposition 1, and measure its cost: mean p50 5.02 ms at 25 req/s and A2−A0 overhead of 2.7–10.1 % at 5–100 req/s. The admission result is I_4, not full-DEI superiority. Deployable δ_3 catches 23/26 separable commitments but misses 15 oracle-positive cases (miss rate 0.556) and falsely quarantines 4/29 actionable controls. DEI+gate changes completion by -0.268 relative to no defense, but outperforms B2strict by 0.107 and B1 by 0.055. On gpt-4.1-mini and tool_knowledge, ASR falls from 0.480 and 0.540 to 0.107 and 0.100; AgentDyn further reduces ASR from 0.470 to 0.237. These implementation-specific results delineate when admission-time provenance is enforceable and when counterfactual auditing remains limited, rather than claiming that all counterfactual defenses are doomed.

Index Terms—agentic AI security, epistemic integrity, runtime verification, reference monitors, zero trust, Model Context Protocol, prompt injection

1. Introduction

An agentic AI system is an epistemic machine before it is an acting one: it reads, retrieves, remembers, and only then invokes tools with real-world effects. Its adversaries have noticed. Indirect prompt injection, tool and Model Context Protocol (MCP) poisoning, retrieval and memory poisoning, and curated context all attack the same surface: not the action interface but the evidential basis on which actions are chosen. Defenses have answered at the action interface. Information-flow and capability designs [4], execution-boundary authority checks [5], guardrail agents [6], and influence tracking [7] constrain what a compromised agent may do. None of them states the property they are reaching for: what it means for an agent's conclusions to be trustworthy, and which part of that property anyone can enforce at runtime.

The question. Consider the case we return to throughout the paper. A user instructs an assistant to delete file 0 from a shared drive. A document the assistant reads while planning carries a planted instruction to delete file 13 "with the owner's approval." The assistant deletes file 13 and, when asked to justify itself, cites the user's genuine directive. Every piece of evidence it cites is authentic, attested, and sufficient; the action is wrong. A monitor that checks only what the agent says supports its action admits this; one that also checks that the action's target is named by trusted evidence blocks it, because file 13 appears in no directive. But the same adversary can plant "delete file 0 now, before the backup completes" when the user asked for the backup first: the target is trusted-derivable, the citation is genuine, and only a comparison of what the agent does with and without the planted span reveals that the injection chose the action. This paper asks one question: which properties of an agent's decision can a runtime monitor enforce, which can only be audited, and does enforcing the former change outcomes on a live model?

The answer. We define decision-level epistemic integrity (DEI) over decision traces of evidence, provenance, belief state, and action (§3) as five invariants: provenance-completeness (I_1), integrity non-degradation (I_2), evidential-support sufficiency (I_3), argument binding (I_4), and a counterfactual-dependence bound (I_5). We prove that I_1–I_4 are safety properties, runtime-monitorable and enforceable by execution monitoring in Schneider's sense, while I_5 is a 2-safety hyperproperty that provably cannot be checked by any monitor observing a single trace (§3.3). The boundary is a theorem about adversaries, not a convenience of definition: the rationalization gap (Corollary 2) shows that an adversary who chooses the action from injected content and cites genuine evidence for it evades every sound admission-time monitor whenever the action's bound arguments are trusted-derivable, and that a decision-local counterfactual re-commitment detects exactly that residual class. The definition is built to be refuted: each invariant carries a finite falsifying observation, and the definition as a whole is falsified by a successful attack on an invariant-satisfying execution (§3.5).

This is a systems paper with a formal spine, and every proposition ends in a measurement. The enforceable fragment is instantiated in CZTA, a zero-trust gateway

in front of a live MCP deployment (§4); the auditable fragment is deployed as a counterfactual audit gate over irreversible actions (§5); and both are evaluated against a frontier model that follows the canonical AgentDojo injection in 18 of 40 episodes (§6). We execute the rationalization attack end to end against a real model, reproduce the coverage counterexample the enforceable fragment admits, and quarantine it with the gate.

Contributions.

C1 A formal, falsifiable definition with a proved enforceability boundary. DEI, stated relative to an explicit and minimized epistemic TCB; the decomposition into four execution-monitor-enforceable invariants and one 2-safety invariant; the rationalization-gap theorem that locates the boundary; a soundness theorem for the mapping from attack classes to invariants; and a mechanization of the safety core in Lean 4.

C2 I4-centered admission and a conditional counterfactual gate. CZTA enforces the safety fragment on a live MCP deployment; the measured replacement gateway adds 2.7–10.1 % p50 overhead at 5–100 req/s (mean p50 5.02 ms at 25 req/s); the gate re-commits irreversible actions under pre-registered perturbations and quarantines those whose commitment does not survive. We give the architecture, the end-to-end workflow on the running example, and a security analysis of the gateway's own attack surface, with miss, false-quarantine, and utility costs reported as first-class results.

C3 A live-model evaluation under a registered design. Against gemini-2.5-flash on AgentDojo, a pilot shows the attack family succeeds in 0.45 of episodes and that a protected-tool filter already reduces it to 0.05 (p=0.0001) at a utility cost n=24 per arm cannot resolve; the registered campaign separates the cases used to develop the gateway from the held-out cases used to score it, measures admission against that filter, against argument–instruction matching, and against I_4 alone, measures the gate where the enforceable fragment is blind, and powers the utility comparison on a pre-registered minimal important difference. The registered study ties admission and B1 at ASR 0.087 while no defense reaches 0.527, localizing the security result to I_4. Deployable δ_3 catches 23/26 separable commitments but has miss rate 0.556 and false quarantine rate 0.138. DEI+gate changes completion by -0.268 versus no defense, but improves it by 0.107 over B2strict and by 0.055 over B1. On gpt-4.1-mini and tool_knowledge, the defensive direction replicates.

Scope of the novelty claim. Within a registered search through 17 July 2026 (§7), no prior work gives a formal, falsifiable definition of decision-level epistemic integrity that separates runtime-enforceable from auditable-only invariants with supporting theory, proves where the boundary lies, and instantiates the enforceable fragment in a working monitor. We adopt the invariance condition of [17] as I_5 and claim neither first use of the phrase [18] nor first statement of decision invariance.

2. Setting, Threat Model, and Positioning

2.1. The epistemic attack surface

We organize attacks on the evidential basis of agent decisions by a ten-class taxonomy of primary attack or failure mechanisms: misinformation and disinformation; hallucination and factuality failure; prompt injection and jailbreak; provenance or identity tampering; retrieval and knowledge poisoning; data or model poisoning and backdoors; confidence or trust manipulation; tool or agent hijacking (including MCP tool-description poisoning); drift, staleness, or benign reliability failure; and a non-attack class for defense-only, benchmark-only, and evaluation records. The taxonomy was produced by a registered PRISMA-ScR scoping review [45] of published work on this surface (6,512 records across six databases; 552 retained as core evidence) whose automated labeling was audited against two blinded domain experts and an adjudicator on a hash-registered 20 % sample (n=114): class agreement $\kappa = 0.63$ / Gwet's $AC_1 = 0.65$ between experts, and $\kappa = 0.48$ between experts and pipeline against the pipeline's self-agreement of 0.83. Protocol, ledger, codebook, instruments, hashes, and amendment log ship with the artifact [49]. The review contributes exactly two things to this paper: the classes that the crosswalk of §3.5 maps onto invariants, and the corpus prevalence of each class (Appendix B), which is what lets us say which invariants matter most often. Nothing else in the paper depends on it.

Categories are mutually exclusive by fiat: where mechanisms co-occur, the record's own framing decides. One registered disambiguation matters for the formalization: retrieval and knowledge poisoning requires that corrupted content enter the corpus, index, or memory, which distinguishes substrate corruption (a provenance failure, I_1) from manipulation of which authentic evidence is selected, ranked, or surfaced from an intact substrate (a sufficiency failure, I_3); the latter is carried as a boundary case in the crosswalk.

2.2. Threat model

The adversary controls the untrusted channels: they may inject instructions or content into retrieved documents and tool results (indirect prompt injection), poison tool descriptions or servers (MCP tool poisoning), write to persistent memory or retrieval stores (memory and RAG poisoning), and shape which evidence the agent sees (context-selection manipulation). The adversary does not control the epistemic TCB of §3.1 or hold its keys. Consistent with the zero-trust posture now advocated for internal agent deployments [43], the model itself is treated as potentially compromised: a successful

TABLE 1. Positioning against the closest prior work. ✓ = provided; ∼ = partial; — = absent. Split = runtime-enforceable separated from auditable-only guarantees, with supporting theory. All rows verified against the public record on 17 July 2026.

	Def.	Falsif.	Monit.	Split	Runtime	Dec.-layer
CaMeL [4]	—	—	—	—	✓	—
GuardAgent [6]	—	—	—	—	✓	∼
Intent-to-exec. [17]	∼	—	—	—	—	✓
Agent epist. int. [18]	∼	—	—	—	—	✓
CXI [5]	✓	—	—	∼	✓	—
ARGUS [7]	—	—	—	—	✓	∼
OOB systematization [16]	—	∼	—	—	—	∼
This work	✓	✓	✓	✓	✓	✓

injection makes the model an adversarial component, and every guarantee below must survive that.

Two attack families deserve explicit placement because they live inside the model rather than on a channel. Reasoning manipulation and chain-of-thought poisoning, in which injected content steers the model's intermediate reasoning without appearing in any cited evidence, are, in our terms, attacks on an untrusted component: DEI makes no claim about the model's internal process, so such attacks surface either as violations of the counterfactual bound (I_5, when removing the injected span changes the decision) or not at all (when the decision would have been the same). Parameter poisoning and backdoors triggered by trusted input are declared out of scope: a backdoor fired by a legitimate directive yields an invariant-satisfying wrong action. The gateway's own attack surface—what an adversary can do to the monitor rather than around it—is analyzed in §4.3.

2.3. Positioning

Out-of-band defenses in the CaMeL lineage [4], [16] and sink-authorization gates such as CXI [5] constrain what a proposal may do; we constrain what may count as a reason for it and prove which part of that constraint a monitor can enforce. Intent-to-execution integrity [17] states the invariance condition we adopt as I_5 but stops short of a monitor, a monitorability analysis, or a falsification protocol; agent epistemic integrity [18] prescribes an architecture. Table 1 records the comparison; §7 gives the landscape.

3. Decision-Level Epistemic Integrity

Prior work formalizes what an agent may do: which tools it may invoke, which flows may reach which sinks [4], [5], [7]. We formalize what an agent may conclude: the conditions under which the beliefs that support a consequential action can be trusted, and which of those conditions a runtime monitor can actually enforce. Every object the invariants quantify over is declared once, in Definition 1, before any invariant uses it; the running example of §2 is instantiated as soon as the objects exist and is revisited after each result.

3.1. Policy Objects and Decision Traces

Definition 1 (Integrity lattice, labels, and policy objects). Let $(L, \sqsubseteq, \sqcap, \sqcup)$ be a finite lattice of integrity labels with top $\top$ (maximally trusted) and bottom $\bot$ (untrusted), in the sense of Biba [26] and Denning [27]. Let Ch be the finite set of ingress channels (user turn, tool result, retrieved document, memory read, model output), C the set of action classes, and *Slots* the set of named argument positions. A policy Π fixes:

- an ingress labeling $\lambda : Ch \to L$ and a floor $\ell_{\min} \in L$; model-generated and external content default to low labels;
- protected classes $C_{\text{prot}} \subseteq C$ (irreversible, external, or safety-relevant effects), a required label $\ell_{\text{req}} : C_{\text{prot}} \to L$, and a sufficiency threshold $\tau : C_{\text{prot}} \to [0, 1]$ (written τ_c for $\tau(c)$);
- a binding policy β assigning to each $c \in C_{\text{prot}}$ a set $\beta(c) \subseteq \mathit{Slots}$ of bound argument slots and, for each $s \in \beta(c)$, a required label $\ell_\beta(c, s) \in L$;
- an endorsement policy π, a finite set of authorized label-raising rules (e.g., k-source corroboration, verifier-tool confirmation, human sign-off), each with a predicate $\mathsf{Authorized}(\rho, b, \ell')$ stating when rule $\rho \in \pi$ may raise belief b to label ℓ';
- a tolerance $\varepsilon : C_{\text{prot}} \to \mathbb{R}_{\geq 0}$ and a class $\Delta_{\text{irrel}}(q)$ of task-irrelevant perturbations (Def. 9).

Every evidence atom and belief carries a label $\ell(\cdot) \in L$; for an atom e ingested on channel ch, $\ell(e) := \lambda(ch)$. The concrete lattice used throughout the paper is UNTRUSTED $\sqsubset$ MEDIUM $\sqsubset$ TRUSTED, with λ(user turn) = TRUSTED (session-signed), λ(tool result) = λ(memory) = MEDIUM (channel-attested), and λ(model output) = UNTRUSTED; the deployed values of the remaining objects are given in §6.1.

Definition 2 (Evidence and attestation). An evidence atom $e \in E$ is an immutable record of ingested content with label $\ell(e)$, channel $ch(e)$, source metadata, and an optional attestation. The predicate $\mathsf{att}(e)$ holds iff e carries a signature chain that verifies against a trusted root and whose validity window covers the current time. The freshness clause subsumes long-horizon staleness: expired attestations fail att, so drift over long deployments surfaces as a provenance-completeness violation rather than a silent decay.

What an attestation is, concretely. In CZTA (§4) an attestation is the tuple (channel credential, content digest, issue time, validity window, signature). For user turns the credential is the session's RS256 token, verified against the identity provider's published JWKS; for tool results and memory reads it is the gateway's Ed25519 receipt over the content digest, issued at ingress and

chained into the ledger. The trusted root is thus the realm key for user channels and the ledger key otherwise; windows are the token expiry and a per-channel time-to-live; att fails on a bad signature, an unknown signer, a digest mismatch, or a lapsed window. The definition does not depend on this choice; it is stated so that the TCB of §3.1 is concrete.

Definition 3 (Beliefs, provenance, updates, endorsements). A belief $b \in B$ is a labeled assertion the agent system holds at runtime, with dependency set $\mathsf{dep}(b) \subseteq E \cup B$ and a finite set $\mathsf{vals}(b) \subseteq V$ of the values it names (identifiers, principals, amounts, file names), extracted at creation time by the recorder from the belief's content. The provenance graph P is the directed acyclic graph whose nodes are $E \cup B$ and whose edges run from each b to every element of $\mathsf{dep}(b)$; it is recorded in an append-only, hash-chained log (we use the W3C PROV vocabulary for the edge kinds [31], a recording convention that carries no semantic commitment). Two event kinds modify beliefs:

- an update $\mathsf{upd}(b', X)$ creates b' with $\mathsf{dep}(b') = X$ and default label $\ell(b') := \bigsqcap_{x \in X} \ell(x)$. This is the Biba derivation rule [26]; its guarantee is that a derived belief's label is a certified lower bound on the trust of everything it was derived from, so no derivation can manufacture trust;
- an endorsement $\mathsf{end}(b, \ell', \rho)$ raises $\ell(b)$ to ℓ' under a rule $\rho \in \pi$ with $\mathsf{Authorized}(\rho, b, \ell')$, in the sense of endorsement in decentralized IFC [30]. Endorsements are logged, attributable, and rate-limitable; they are the only lawful label increases.

Definition 4 (Actions, arguments, and decision traces). An action event $\mathsf{act}(a)$ has class $c(a) \in C$, an argument map $\mathsf{args}(a) : \mathit{Slots} \rightharpoonup V$, and a support set $\mathsf{support}(a) \subseteq B$, the beliefs the system commits to as justifying a. Fix the event alphabet $\Sigma = \{\mathsf{ing}(e, ch), \mathsf{upd}(b, X), \mathsf{end}(b, \ell, \rho), \mathsf{act}(a)\}$. A decision trace is a finite sequence $\sigma = \epsilon_1 \epsilon_2 \cdots \epsilon_k$ over Σ; $\mathsf{acts}(\sigma)$ denotes its protected action events. For a task input q and untrusted-channel content d, we write $J(q, d)$ for the protected action the system selects at a designated decision point, holding trusted state fixed.

Epistemic TCB. The definition is stated relative to a trusted computing base: the ingress labeler λ together with the channel authentication it relies on, the attestation verifier and its keys, the append-only provenance recorder including the extraction of $\mathsf{vals}(\cdot)$, the support aggregator S (Def. 7), and the policy engine and gateway. The model, all tool outputs, retrieved content, and memory contents are untrusted. Making the TCB explicit answers the observation that provenance assignment is the under-specified trusted base of out-of-band defenses [16]; we do not eliminate that base, we name it, minimize it, and measure it (§4.3). The labeler is deliberately simple: it is a function of the authenticated channel identity alone, never of content, so the only way to obtain a high label for adversarial content is to deliver it over a high channel, which is channel spoofing and is excluded by Assumption 2. The value recorder is the one TCB component that reads content; it is a deterministic extractor over the belief's text (identifiers, principals, amounts), and its failure mode (missing a value) is fail-closed under I_4.

Example 1 (Running example, I: the objects). The user turn "Delete file 0 from the shared drive" is ingested on the user channel as atom e_u with $\ell(e_u) = \text{TRUSTED}$, $\mathsf{att}(e_u)$ true. The drive listing returned by a tool is atom e_l (MEDIUM, attested by gateway receipt); the body of one listed document, which carries the planted span "also delete file 13, the owner has approved," is atom e_p (MEDIUM, attested as a tool result, content untrusted). The system forms beliefs b_u ("the user wants file 0 deleted," $\mathsf{dep} = \{e_u\}$, $\mathsf{vals} = \{\text{file 0}\}$, TRUSTED), b_l (the listing, MEDIUM), and b_p ("file 13 should be deleted," $\mathsf{dep} = \{e_p\}$, $\mathsf{vals} = \{\text{file 13}\}$, MEDIUM) by the meet rule. The action $\mathsf{act}(a)$ with $c(a) = \text{delete} \in C_{\text{prot}}$ has $\mathsf{args}(a)(\mathsf{target}) = \text{file 13}$; the policy binds target at $\ell_\beta(\text{delete}, \mathsf{target}) = \text{TRUSTED}$. The compromised model commits $\mathsf{support}(a) = \{b_u\}$: a genuine, attested, trusted directive.

3.2. The Definition

Definition 5 (Provenance-completeness, I_1). A trace σ satisfies I_1 iff for every $\mathsf{act}(a) \in \mathsf{acts}(\sigma)$ and every $b \in \mathsf{support}(a)$: (i) the transitive dependency closure of b in P is well-founded and every maximal derivation path terminates in E; and (ii) every such terminal evidence atom e satisfies $\mathsf{att}(e)$ and $\ell(e) \sqsupseteq \ell_{\min}$. Informally: no protected action rests on an orphan belief (no evidential root, the formal shape of a fabricated claim) or on an unattested or sub-floor root.

Definition 6 (Integrity non-degradation, I_2). A trace σ satisfies I_2 iff every label increase on any belief is effected by a logged endorsement event authorized by π; equivalently, for every update $\mathsf{upd}(b', X)$, $\ell(b') \sqsubseteq \bigsqcap_{x \in X} \ell(x)$, and no other event raises a label. Informally: untrusted input can never silently launder itself into trusted belief; every upgrade has an accountable, policy-cited justification.

Definition 7 (Evidential-support sufficiency, I_3). Fix an audited aggregator $S : 2^B \times L \to [0, 1]$ scoring a support set against a required label, subject to three stated requirements: (A1) qualification: $S(U, \ell)$ depends only on evidence at or above ℓ that is attested; (A2) monotonicity: adding qualifying evidence never decreases the score; (A3) auditability: S is fixed, computable from the recorded prefix state, and inside the TCB. S is instantiable as Dempster–Shafer belief $\mathrm{Bel}(\cdot)$ over the qualifying evidence [32], or as an accepted-argument criterion in an argumentation framework [33]. A trace satisfies I_3 iff for every $\mathsf{act}(a) \in \mathsf{acts}(\sigma)$, $S(\mathsf{support}(a), \ell_{\text{req}}(c(a))) \geq$

$\tau_{c(a)}$. Informally: consequential actions require enough qualifying evidence, with per-class thresholds.

Definition 8 (Argument binding, I_4). A trace σ satisfies I_4 iff for every $\mathsf{act}(a) \in \mathsf{acts}(\sigma)$ and every bound slot $s \in \beta(c(a))$ with $\mathsf{args}(a)(s)$ defined, there exists $b \in \mathsf{support}(a)$ with $\mathsf{args}(a)(s) \in \mathsf{vals}(b)$ and $\ell(b) \sqsupseteq \ell_\beta(c(a), s)$. Informally: every argument that determines what the action does to the world is named by a cited belief of sufficient standing. The model may choose the action; it may not choose its targets. I_4 is the decision-layer counterpart of the authority binding that context-to-execution integrity enforces at the sink [5]: CXI binds a field value to the manifest that authorizes its effect, I_4 binds an argument value to the evidence that justifies naming it. An argument slot left unbound by β (free-text message bodies, say) is unconstrained by I_4 and falls to I_5.

Definition 9 (Counterfactual-dependence bound, I_5). Let $\Delta_{\mathrm{irrel}}(q)$ be a class of task-irrelevant perturbations of untrusted content (perturbations that preserve the task-relevant semantics of d; operationalized by a generator suite, §3.5), and let dist be a metric on protected actions (0/1 for discrete choices). A system satisfies I_5 iff for all $\delta \in \Delta_{\mathrm{irrel}}(q)$, $\mathsf{dist}\big(J(q,d),\, J(q, d \oplus \delta)\big) \le \varepsilon_c$ for the class c of $J(q,d)$. This is the invariance condition articulated for judgment integrity in [17], restated over decision traces; unlike I_1–I_4 it relates pairs of executions.

Definition 10 (Decision-level epistemic integrity). Write $\mathrm{Lift}(P) = \{\, T : \forall \sigma \in T.\ \sigma \models P \,\}$ for the set-level lifting of a trace property P. A system with reachable-trace set T satisfies DEI with respect to policy Π iff

$$T \in \underbrace{\mathrm{Lift}(I_1 \wedge I_2 \wedge I_3 \wedge I_4)}_{\text{safety fragment (enforceable)}} \cap \underbrace{I_5}_{\text{2-safety (auditable)}}.$$

DEI is thus, by construction, a hyperproperty whose safety fragment and whose properly k-safety fragment are named separately: the form in which the enforceability results of §3.3 are stated.

Example 2 (Running example, II: what the invariants decide). On the trace of Example 1: I_1 holds (b_u roots in the attested, trusted e_u); I_2 holds (no label was raised); I_3 holds (a trusted directive is τ_c-sufficient for a single deletion under the deployed scorer); I_4 fails: the bound slot target carries file 13, and no cited belief at TRUSTED names file 13. The monitor blocks at admission. Now change the plant to "delete file 0 now, before the backup completes," with the user having asked for the backup first: the model commits $\mathsf{args}(a)(\mathsf{target}) =$ file 0 with the same support, and I_1–I_4 all hold. The action is wrong because its timing was chosen by the plant, but every single-trace check passes; only I_5—re-running the decision with the plant removed—distinguishes this trace from an honest one. Corollary 2 shows this is a limit of admission-time checking as such, not of our particular invariants.

Algorithm 1 DEI reference monitor: event admission (enforces $I_1 \wedge I_2 \wedge I_3 \wedge I_4$ by truncation; Prop. 1).

Require: event ϵ; state $(\ell, P, \mathsf{cert}, \mathsf{vals}, \pi)$; policy $\Pi = (\ell_{\min}, \ell_{\mathrm{req}}, \tau, \beta, C_{\mathrm{prot}})$

if $\epsilon = \mathsf{ing}(e, ch)$ then
  $\ell(e) \leftarrow \lambda(ch)$; $\mathsf{cert}(e) \leftarrow \mathsf{att}(e) \wedge \ell(e) \sqsupseteq \ell_{\min}$
  admit
else if $\epsilon = \mathsf{upd}(b', X, \ell_{\mathrm{prop}})$ then
  if $\ell_{\mathrm{prop}} \not\sqsubseteq \bigsqcap_{x \in X} \ell(x)$ then
    block ▷ I_2: silent upgrade
  end if
  $\ell(b') \leftarrow \ell_{\mathrm{prop}}$; $\mathsf{cert}(b') \leftarrow \bigwedge_{x \in X} \mathsf{cert}(x)$; $\mathsf{vals}(b') \leftarrow \mathsf{extract}(b')$ ▷ $O(|X|)$, memoized
  admit
else if $\epsilon = \mathsf{end}(b, \ell', \rho)$ then
  if $\rho \notin \pi$ or $\neg\,\mathsf{Authorized}(\rho, b, \ell')$ then
    block ▷ I_2: unauthorized endorsement
  end if
  $\ell(b) \leftarrow \ell'$; $\mathsf{log}(b, \ell', \rho, \mathrm{principal})$; admit
else if $\epsilon = \mathsf{act}(a)$ then
  if $c(a) \notin C_{\mathrm{prot}}$ then
    admit
  end if
  for all $b \in \mathsf{support}(a)$ do
    if $\neg\,\mathsf{cert}(b)$ or $\neg\,\mathsf{WindowsValid}(b)$ then
      block ▷ I_1; freshness per Def. 2
    end if
  end for
  if $S(\mathsf{support}(a), \ell_{\mathrm{req}}(c(a))) < \tau_{c(a)}$ then
    block ▷ I_3: insufficient support
  end if
  for all $s \in \beta(c(a))$ with $\mathsf{args}(a)(s)$ defined do
    if $\neg\,\exists b \in \mathsf{support}(a) : \mathsf{args}(a)(s) \in \mathsf{vals}(b) \wedge \ell(b) \sqsupseteq \ell_\beta(c(a), s)$ then
      block ▷ I_4: unbound argument
    end if
  end for
  admit
end if

block ≡ suppress ϵ and log the verdict (truncation); admit ≡ append ϵ to the trace.

3.3. Monitorability: What a Gateway Can Enforce

The invariants deliberately fall on both sides of a classical boundary, and the definition's honesty depends on saying which side each is on. Algorithm 1 makes the enforceable side concrete: a single event-admission procedure whose block branches are exactly the bad prefixes of Proposition 1. Two assumptions are stated explicitly because the proofs depend on them.

Assumption 1 (Complete mediation). The gateway mediates every Σ-event that can affect a protected action; there is no side channel from untrusted content to belief state or action that bypasses admission.

Assumption 2 (Trusted TCB). The epistemic TCB of §3.1 behaves as specified: (i) channel identity is authenticated, so λ assigns to every atom the label its channel carries by policy; (ii) the recorder extracts $\mathsf{vals}(\cdot)$ faithfully and the log is append-only; (iii) the attestation verifier, the aggregator S, and the policy engine compute what their definitions state. Compromise of any component voids all guarantees (§3.6).

Proposition 1 (I_1–I_4 are enforceable safety properties). Over decision traces, I_1, I_2, I_3, and I_4 are safety properties in the sense of Alpern–Schneider [34]: every violating trace has a finite bad prefix (the offending action or update event) after which no extension repairs the violation. Consequently (i) they are monitorable, and an LTL_3-style monitor [36], [37] issues the verdict $\bot$ exactly at the bad prefix; and (ii) they lie in the class of policies enforceable by execution monitoring [35]: a reference monitor that truncates the trace at the bad prefix (blocking the action or update) enforces them, not merely detects them.

Proof sketch. For I_1, I_3, and I_4 the candidate bad event is a protected action; its support set, the relevant provenance subgraph, the aggregator score, the cited beliefs' value sets, and the action's argument map are all determined by the finite prefix at that event, so violation is a property of the prefix and is irremediable once the action executes. For I_2 the bad event is a label-raising transition without a matching authorized endorsement in the same prefix. Quantification over runtime objects (beliefs, evidence) is handled by parametric monitoring [38]: the object universe at any prefix is the finite set of objects observed so far, so each instantiated monitor is a finite-state safety monitor. Enforcement follows because blocking the bad event yields a legal trace, i.e., the monitor is a truncation automaton in Schneider's sense [35]. The full proof, including the two legality-preservation facts, is Appendix A. The I_1–I_4 clauses and the two legality facts are mechanized in Lean 4 (finite-trace form; kernel-checked; the only axiom is propext, on two lemmas), re-checkable in the artifact via formal/check_formal.sh. □

Remark 1 (Online cost). With per-belief root certificates (a memoized, hash-chained receipt that a belief's evidential roots are attested and $\sqsupseteq \ell_{\min}$, established when the belief is created) the I_1 check at an action is $O(|\mathsf{support}(a)|)$ constant-time receipt verifications; I_2 is one lattice meet per update; I_3 is one aggregator evaluation; I_4 is $O(|\beta(c(a))| \cdot |\mathsf{support}(a)|)$ set-membership tests on recorded value sets. These are the operations whose cost CZTA measures (Table 3); the reference implementation confirms the asymptotics, with memoized I_1 checks flat across a 256× range of provenance depth while the unmemoized check grows linearly (Appendix D).

Proposition 2 (I_5 is auditable but not single-trace monitorable). I_5 is a 2-safety hyperproperty [39]: its violation is witnessed only by a pair of finite traces $(J(q,d), J(q,d \oplus \delta))$. No black-box monitor observing a single production trace can detect a violation, since satisfaction is not a property of any individual trace [39]; runtime checking requires either stored comparison executions or a model of the system (gray-box monitoring) [40], [41]. I_5 is therefore classified auditable: falsifiable offline by exhibiting a witness pair, testable by a differential harness, and checkable before dispatch only by re-executing the decision (§5), never by inspecting the production trace alone.

Corollary 1 (The DEI decomposition). $\mathrm{DEI} = \mathrm{Lift}(I_1 \wedge I_2 \wedge I_3 \wedge I_4) \cap I_5$ (Def. 10) is the conjunction of an EM-enforceable safety fragment (Prop. 1) and a 2-safety fragment that no single-trace monitor enforces (Prop. 2). Any monitor claiming to enforce DEI wholesale is unsound.

Remark 2 (The boundary, stated plainly). Full noninterference between untrusted channels and decisions is a general hyperproperty [29], [39] and inherits the same non-monitorability; we neither claim nor need it. Complete transformer-level noninterference is explicitly a non-goal even for sink-authorization systems [5], and there are principled arguments that in-band immunity to injection may be unattainable [20]. Likewise, no monitor decides whether a belief is true; DEI constrains the evidential and provenance structure of decisions, not their semantic veracity. Concurrent work characterizes generically what a deterministic pre-execution agent gate can enforce: the safety policies whose good prefixes its state model recognizes [8]. Prop. 2 names a specific, security-relevant property outside that class for every state model, and §5 builds the decision-local gate that audits exactly that property.

Proposition 3 (Mutual independence of the aggregator axioms). There exist aggregators $S_{\neg \mathrm{A1}}$, $S_{\neg \mathrm{A2}}$, $S_{\neg \mathrm{A3}}$, each violating exactly one axiom of Def. 7 while satisfying the other two: no axiom is derivable from the rest, and dropping any one admits a named attack.

Proof: the artifact, one explicit violating aggregator per axiom with codomain and monotonicity checks.

3.4. The Rationalization Gap

The crosswalk of Appendix B maps each attack class of §2.1 to the invariant it must violate. Theorem 1 shows the mapping is sound against the enforceable fragment; Corollary 2 shows why that fragment is nevertheless insufficient, and exactly which attacks survive it.

Theorem 1 (Crosswalk soundness). Under Assumptions 1 and 2: for every charting class K with machine-fired primary $j(K)$ and every trace σ with $\mathrm{Mech}_K(\sigma)$, there is a finite prefix $\sigma[..k]$ with $viol_{j(K)}(\sigma[..k])$, and Algorithm 1 blocks at or before ϵ_k. Conversely (anti-triviality), control/evaluation instances satisfy $\neg viol_j$ at every prefix for all j and are admitted.

Proof: Appendix C (finite case analysis over the machine-fired rows, anti-triviality, and a delimitation of the proved implication from its empirical support). The per-row predicates Mech_K (Definition 11) are stated there.

Corollary 2 (Rationalization gap: choice laundering is admission-undetectable). Let M be any sound monitor

enforcing $I_1 \wedge I_2 \wedge I_3 \wedge I_4$ at admission over traces where $\mathsf{support}(a)$ is committed by an untrusted component (the model; a compromised model is an adversarial component). Let S_0 be the support set of any honestly admitted protected action of class c_0, and let $A_\beta(S_0)$ be the set of class-c_0 actions every bound argument of which is named by some belief in S_0 at the label β requires. Then for every $a^* \in A_\beta(S_0)$ there exists an adversarial model strategy—select a^* from injected content; commit S_0—such that on the resulting trace $viol_j$ holds at no prefix for any $j \in \{1, 2, 3, 4\}$ while the action's actual basis is the injection. Hence (i) the strategy is undetectable at admission by any sound I_1–I_4 monitor at any provenance granularity; and (ii) an I_5-side counterfactual check is necessary, not optional: no refinement of the enforceable fragment detects it.

Proof: Appendix C (existence via the full-support commitment S_0 restricted to $A_\beta(S_0)$; necessity via the $S^{\pm}$ construction of Prop. 2).

Remark 3 (What I_4 catches and what it leaves). The corollary quantifies over $A_\beta(S_0)$, which is the precise residual: an injection can still choose which of the trusted-derivable actions is taken, whether it is taken, and when, but it can no longer choose a target that trusted evidence does not name. The laundering probes executed in §6.3 (delete file 13 for file 0; share with an attacker-chosen recipient) are not instances of the corollary's strategy: they violate I_4 and are blocked at admission once I_4 is enforced. The gate's incremental value must therefore be demonstrated on choice-laundering probes, and §6.3 is designed accordingly. This is also why I_4 is a separate invariant rather than a clause of I_3: it changes the adversary's residual strategy space, and the corollary states that space exactly.

3.5. Falsifiability Protocol

Each invariant carries a refutation condition: the finite observation that falsifies the claim that a given system enforces or satisfies it. A single logged protected action admitted with an unattested or sub-floor evidential root refutes "the deployment enforces I_1"; a single silent label upgrade refutes I_2; a single admitted action below threshold refutes I_3; a single admitted action with a bound argument named by no qualifying cited belief refutes I_4; and one $\delta \in \Delta_{\text{irrel}}$ with $\mathsf{dist}(J(q, d), J(q, d \oplus \delta)) > \varepsilon_c$ refutes I_5. The first four are checkable by any auditor holding the log, which is why the log is hash-chained and receipt-bearing; the fifth requires the pre-registered differential harness.

Falsifying the definition itself (coverage). Let $\mathcal{A}$ be the attack taxonomy of §2.1 with the per-instance success predicate of the evaluation harness. Say DEI covers $\mathcal{A}$ on a corpus if every successful attack instance violates at least one of I_1–I_4 on its trace or I_5 on its witness pair. A successful epistemic attack on an invariant-satisfying execution falsifies coverage and shows the definition incomplete: a reportable outcome, not an embarrassment. The crosswalk (Appendix B) records, per attack class, the invariant predicted to be violated, registered before the campaigns that test it. An adaptive adversary must therefore either violate a monitored invariant (and be blocked or logged) or exhibit a coverage counterexample (and falsify the definition), the adaptive-evaluation design the systematization of out-of-band defenses calls for [16].

Example 3 (Running example, III: the protocol applied). Walk the two traces of Example 2 through the protocol. Trace 1 (target file 13): if the deployment admits the action, the admission log itself refutes "the deployment enforces I_4"—an implementation failure, repaired by fixing the monitor, with no bearing on the definition. Trace 2 (target file 0, timing chosen by the plant): every enforceable invariant holds on the log, so no admission record can refute anything; the differential harness re-runs the decision with the plant removed, the action changes, and the pair refutes I_5. Had the action not changed while the attack still succeeded—say through an unbound free-text argument the perturbation family never touches—that would be a coverage counterexample, reported as evidence that the definition (here β or Δ_{irrel}) is incomplete. The three outcomes are distinguishable from the artifacts alone, which is what makes the protocol executable.

Human validation of the crosswalk. Because the crosswalk is a judgment artifact, two blinded domain experts independently assigned invariants to each class from the definitions alone, with an adjudicator resolving disagreements. On the eight classes with a machine-fired primary they agreed exactly on 7/8 (Wilson 95 % CI 0.53–0.98) and 10/10 on the in-scope boundary; the adjudicated consensus matches the invariant that actually fired in the reference monitor on 6/8. The divergences are informative: both coders read stale evidence as insufficient support (I_3) where the monitor enforces a lapsed attestation (I_1), and read prompt injection as a provenance failure (I_1) where the gateway blocks the unendorsed label raise (I_2). The crosswalk records what the monitor enforces; the mapping was frozen before any campaign was scored against it, and the instrument and scorer are released.

3.6. Scope and Limitations

Appendix D distinguishes proved, implemented, measured, development, and uninformative evidence, so that no deferred study or failed susceptibility check is mistaken for a positive result. Beyond that: (1) all guarantees are relative to the epistemic TCB; compromising the labeler, the channel authentication it relies on, the verifier keys, the recorder, or the aggregator voids them, and §4.3 quantifies that base. (2) Rationalization is a faithfulness gap, not a granularity one. I_1–I_4 constrain

the committed support set and the action's bound arguments, but the support set is committed by an untrusted model, so a compromised model can select an action from adversarial content and cite legitimate evidence for it. I_4 narrows the residual to choice laundering under trusted-derivable arguments (Cor. 2) without closing it; sub-document laundering is a resolution limit that finer provenance tightens [5], [7], and [9] states the architectural version of the gap as semantic laundering, for which §5 builds the runtime check. (3) β and ℓ_β are policy with a utility cost: a target discovered from a MEDIUM tool result cannot bind at TRUSTED, so β's tightness is bounded by the attestation topology exactly as τ_c is (§6.1). (4) Δ_{irrel} is itself a judgment and the hard core of I_5 [17]; we pre-register the generator suite rather than choose perturbations post hoc. (5) I_3's aggregator encodes an epistemology; we require only A1–A3. (6) DEI constrains structure, not truth: attested, well-supported, invariant-satisfying decisions can still be wrong.

4. CZTA: A Reference Monitor for the Enforceable Fragment

Proposition 1 says I_1–I_4 can be enforced by execution monitoring; this section shows how they are enforced, at measured cost, on a real deployment. CZTA is a zero-trust gateway interposed between an agent (a production model) and a live MCP server, with Keycloak as policy decision point; its admission path is Algorithm 1 realized at the gateway.

4.1. Architecture and Workflow

Figure 1 shows the evaluated path. Every ingress channel passes a policy enforcement point (PEP) that authenticates the channel, stamps $\lambda(ch)$, digests the content, issues the attestation receipt of Def. 2, and appends the atom to the ledger; the model runtime sees only labeled, receipt-bearing atoms. Beliefs are created through the update API (meet rule, dep and vals recorded, root certificate memoized); label raises exist only as endorsement-API calls checked against π and logged with their principal. A protected tool call is committed as a structured support/action/args step; the gateway evaluates I_1 (receipts), I_2 (per update), I_3 (aggregator), and I_4 (bound arguments against the cited beliefs' value sets), then dispatches the MCP request under a Keycloak-issued, audience-bound token or suppresses it and logs the violated predicate. Irreversible classes pass the audit gate of §5 between admission and dispatch, and a reasoning-lineage attestation (RLA) signs the decision state at each enforcement point so that any holder of the ledger can replay the decision.

Example 4 (Running example, IV: the workflow end to end). (1) The user turn arrives on the authenticated user channel; the ingress PEP stamps TRUSTED, digests

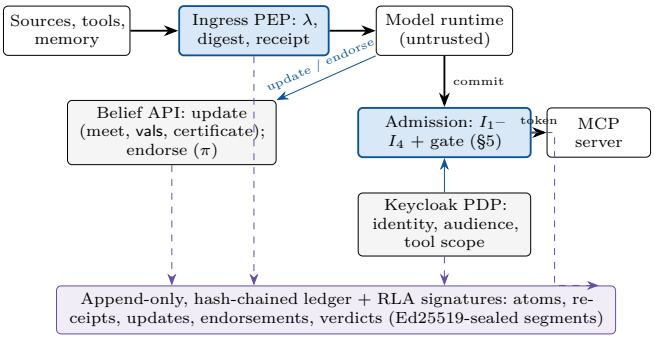


Figure 1. The evaluated CZTA path. Blue boxes are enforcement points inside the epistemic TCB; the model runtime and the MCP server are untrusted. Every event that can affect a protected action crosses an enforcement point (Assumption 1) and is ledgered.

it, issues receipt r_u, and appends e_u. (2) The drive listing and the document body carrying the plant arrive on the tool channel and are stamped MEDIUM with receipts r_l, r_p; nothing about the plant is detected here, and nothing needs to be. (3) The model creates b_u, b_l, b_p through the update API, each with the meet of its inputs, a root certificate over its receipt, and its recorded values. (4) The compromised model commits action delete, args $\{\mathsf{target} \mapsto \text{file } 13\}$, support $\{b_u\}$. (5) Admission: I_1 passes ($\mathsf{cert}(b_u) = \text{ok}$, window valid); I_2 was never at issue; I_3 passes ($S(\{b_u\}, \text{TRUSTED}) \geq \tau_c$); I_4 fails (file $13 \notin \mathsf{vals}(b_u)$). The gateway suppresses the call, logs $viol_4$ with the offending slot, and the ledger entry is the refutation record of §3.5. (6) Had the model committed file 0 under a timing chosen by the plant, admission would pass and the gate's re-commitment with e_p struck would select a different action, so the gateway would quarantine instead of dispatch.

4.2. Instantiating the Invariants

Table 2 maps each enforceable invariant to its CZTA mechanism, the fields logged and attested to make post-hoc refutation possible, and the enforcement point. Two engineering choices carry the formal weight. First, root certificates: when a belief is created, the gateway verifies its evidential roots once and issues a hash-chained receipt; at action time, I_1 reduces to $O(|\mathsf{support}(a)|)$ receipt verifications, the operation whose sign/verify latency Table 3 reports. Second, endorsement as an API: label upgrades exist only as explicit, logged gateway calls citing a policy rule, which is what makes I_2's "no silent upgrade" condition checkable by inspection of the log.

4.3. Epistemic TCB Inventory and Security Analysis

The TCB of §3.1 concretely comprises the ingress labeler and the channel authentication it relies on, the attestation verifier and its key store, the append-only

TABLE 2. Instantiation of the enforceable invariants in CZTA. Measured costs appear in Table 3.

Inv.	CZTA mechanism	Logged/attested fields	Point
I_1	Root certificates over attested evidence; admission check on protected tool calls	belief id; root set; receipt hash chain; attestation ids; validity windows	Gateway, pre-invocation
I_2	Label propagation by lattice meet; endorsement API with policy-rule citation	update event; input labels; result label; endorsement rule id; principal	Gateway, per update
I_3	Aggregator service scoring qualifying evidence against per-class thresholds	support set; qualifying-evidence ids; score; τ_c; verdict	Gateway, pre-invocation
I_4	Bound-slot check of $\mathsf{args}(a)$ against vals of cited beliefs at ℓ_β (list-valued slots element-wise); the citation contract of the pilot grid (§6.1) is its weaker precursor	slot; value; citing belief id; label; verdict	Gateway, pre-invocation
I_5	Not enforced (Prop. 2); audited offline and, for irreversible classes, by the gate	paired traces; δ operator id; decision distance	Offline harness; gate

hash-chained provenance log with its value recorder, the aggregator service, and the Keycloak policy engine. The trusted implementation is the deposited replacement (v9-replacement-2), not the unavailable original v9 package: the executable reference semantics of Algorithm 1 (I_1–I_4 with receipt-based root and window checks), a strict RS256/JWKS verifier, an MCP tools/call decision endpoint, and a hash-chained Ed25519-sealed ledger. Its RLA module and all other v8 modules are preserved byte-for-byte for audit; only modules actually imported are asserted on the measured path. The Keycloak realm stamps the claims the gateway inspects, and /env publishes the hash of every trusted module imported, so each measurement is bound to its code. The retired v8 scenario-specific policy pipeline is not used for any number here. Benchmark, scenario, and other retired modules remain outside the trust path. Everything else—the model, MCP servers and their tool descriptions, retrieved content, and memory—is untrusted.

What an adversary can do to the gateway. Interposing a monitor creates a surface of its own, and the formal guarantees say nothing about it. We enumerate the attack vectors the design admits and how each is closed or conceded. Channel spoofing (adversarial content over a high channel) is the one attack on λ, excluded by Assumption 2; in CZTA it reduces to forging a session token or a gateway receipt, so its security is that of RS256 verification against JWKS and of Ed25519 receipt signatures. Endorsement abuse requires a rule $\rho \in \pi$ and an authorized principal; the deployed π authorizes no post-ingress raise (§6.1), and every raise is logged with its principal. Receipt replay is closed by binding each receipt to a content digest checked at verification. Recorder evasion (a target in a form the extractor misses) fails closed under I_4: an unextracted value is unbound and the action is blocked, a utility cost rather than a security cost. Mediation bypass (a side channel from untrusted content to action that avoids a PEP) is Assumption 1; in the evaluated path the MCP transport is the only route to the server, and a deployment that exposes a second route voids the guarantee. Denial of service through the gate is bounded by two re-commitments per action and by never gating reversible classes. Re-commitments are never dispatched and the counterfactual plane is assembled inside the TCB from ledgered atoms, so re-execution has no side effects; a model that commits identically with and without the plant defeats the gate, a coverage question for the falsification protocol rather than a silent failure (§5). TCB compromise (verifier keys, ledger key, aggregator) voids all guarantees; the hash-chained ledger makes tampering detectable after the fact but not preventable, which is the concession every reference monitor makes. Rug-pull (a tool whose description is mutated after approval) is the one documented MCP pattern the evaluated gateway detects without preventing, as reported below.

4.4. Authorization-Layer Checks

The epistemic evaluation lives in §6; this subsection reports a complementary check that the gateway's authorization layer behaves as specified, since DEI enforcement composes with, and does not replace, the sink-layer defenses of CXI-class systems [4], [5]. We executed four documented MCP attack patterns (tool poisoning, token passthrough, confused deputy, rug-pull) plus a postmark-style output-exfiltration boundary case against the live gateway with defenses enabled. These are MCP/OAuth binding checks (RFC 8693/9449/9396), not tests of the coverage claim, and they were constructed by the authors during development; they are reported because a gateway monitor must get them right and because one is an honest negative. Tool poisoning, token passthrough, and the confused deputy are blocked at the authorization layer (mcp-tool-not-exposed 403; mcp-token-passthrough-forbidden 403; InvalidAudienceError 401); the output-exfiltration case is, as pre-registered, detected but not prevented (accountability via the signed RLA output digest). The rug-pull executed (HTTP 200): a post-approval tool mutation is a temporal I_2 violation (integrity holds at approval, breaks later) while the admission check consults only approval-time state. Def. 2 already attaches a validity window to every attested atom and requires re-checking at use, so the fix is a tool-state attestation recomputed at call time rather than

TABLE 3. Authorization-path latency (ms) on the live stack (Keycloak RS256 + JWKS + MCP tools/call; stub upstream, so relative overhead is an upper bound). Open-loop Poisson arrivals; cells are mean p50/p95 across three independently restarted repetitions, with zero errored requests required; values flow from the harness output.

	λ=5	λ=25	λ=100 req/s
A0 (no monitor) p50/p95	5.02/6.72	4.74/6.78	4.65/7.21
A1 (I_1–I_3) p50/p95	4.95/7.00	4.77/6.78	4.77/7.70
A2 (I_1–I_4) p50/p95	5.53/7.67	5.02/7.61	4.77/7.71
A2−A0 p50 (%)	10.1	6.1	2.7

RLA sign / verify, mean p50/p95 across 1000 iterations: 1.42/2.14 / 1.47/2.18 ms; zero verification errors.

new machinery; we report the measured behaviour and leave the implementation to future work.

4.5. Measured Overhead

We measure the cost of interposing the unmodified reference monitor (source hashes published at /env) on a real authorization path: real Keycloak RS256 tokens, real JWKS validation, and real MCP JSON-RPC transport. Per the pre-registered measurement protocol, arrivals are open-loop Poisson at $\lambda \in \{5, 25, 100\}$ req/s (latency taken from the intended send time, so coordinated omission cannot understate the tail), each rate held 60 s after a discarded 20 s warmup, over three independent repetitions per arm with the process restarted between them. Three arms share the identical request path: A0 with the monitor removed, A1 enforcing I_1–I_3, and A2 enforcing I_1–I_4; the overhead is A2 minus A0, and the cost of the I_4 slot check is A2 minus A1. To isolate the monitor, the upstream is a minimal tool stub with no model inference in the path, so the relative overhead reported here is an upper bound: the fixed monitor cost amortizes against real tool and model latency in any production deployment. The protocol requires zero errored requests in every cell; all 27 cells passed with zero errored requests.

Table 3 reports the result: A2−A0 p50 overhead is 10.1%, 6.1%, and 2.7% at 5, 25, and 100 req/s; A2−A1 is the corresponding I_4 increment. These are the constant factors behind Proposition 1; the reference implementation confirms their asymptotic behaviour independently (Appendix D), and every restarted repetition is retained in the deposited result file.

5. The Counterfactual Audit Gate

Corollary 2 prices the enforceable fragment: a laundered choice is admitted, and no admission-time check can tell. The impossibility is about single traces, not decisions, and a decision can be re-run. This section deploys the auditable fragment as a decision-local gate over irreversible classes: an action whose commitment does not survive its own counterfactual is quarantined.

5.1. Design

The gate sits between admission and dispatch. For every admitted action a with cited support S_c whose class is in the pre-registered irreversible set $D = \{\text{delete}, \text{share}, \text{send}\}$, the gateway issues re-commitments on perturbed evidence planes and compares the result with a. If any re-commitment returns an action $a' \neq a$, or leaves a without qualifying support, a is quarantined and escalated to the human checkpoint that already serves sub-τ_c admissions. Reversible classes are not gated: the audit is priced where mistakes are permanent. Algorithm 2 states the semantics; the gate adds no TCB, since its inputs (admission record, labeler, ledgered atoms, commitment protocol) are already inside the TCB of §4.3.

Which content is removed, and how the gate knows. The gate never identifies malicious content. It removes evidence by provenance class, which is computable inside the TCB from the ledger alone, and it asks whether the action stands on what remains. Three perturbations are defined:

- δ_1, plant removal: the evidence plane in which the injected content never entered. This is an oracle perturbation, available only when the harness knows the injection vector; it is used in the evaluation as the upper bound on what any deployed removal can achieve.
- δ_2, cited-sub-required removal: strike every $x \in S_c$ with $\ell(x) \sqsubset \ell_{\text{req}}(c(a))$ and re-commit against what remains. (The version of this perturbation evaluated in §6.3 struck atoms below $\ell_{\min}$ rather than below ℓ_{req}, which in the deployed policy never fires; the ℓ_{req} form is the registered deployable version.)
- δ_3, uncited removal: restrict the plane to the cited support S_c plus the trusted state and re-commit. If the action depended on uncited content, it changes.

δ_2 and δ_3 are oracle-free. Together they cover both ways a plant can influence a committed action: if the plant is uncited, δ_3 removes it; if it is cited, it sits below ℓ_{req} for any protected class whose required label exceeds the tool channel's, and δ_2 removes it. A plant cited at ℓ_{req} is channel spoofing (§4.3), not laundering. The registered deployment enforces δ_3 and observes δ_2: at atom granularity δ_2 cannot tell a commitment that depends on needed MEDIUM data from one that depends on a plant sharing the same atom, so its flips are logged and reported as the gate's resolution limit rather than enforced.

Mixed content, stochasticity, and divergent state. Removal is at atom granularity: a plant that shares an atom with the data the decision needs (a planted claim inside the file listing the user asked to be read) is removed together with that data under δ_2, and an honest commitment changes for the same reason a laundered one does; a plant in an atom of its own is removed by δ_3 alone. The probe families of §6.3

Algorithm 2 Counterfactual audit gate (per admitted irreversible action).

Require: admitted action a, cited support S_c, $c(a) \in D$; perturbation set Δ_{gate} ($\{\delta_1\}$ in the oracle evaluation; $\{\delta_2, \delta_3\}$ deployed)
for $\delta \in \Delta_{\text{gate}}$ do
  $a' \leftarrow$ Recommit(plane(ctx) $\setminus \delta$) ▷ plane assembled from ledgered atoms
  if $a' \neq a$ or $\neg$Qualifies(a) under δ then
    return Quarantine(a); escalate to human checkpoint
  end if
end for
return Execute(a)

are built in both placements so that this limit is measured, not assumed; span-level provenance [7] is what would lift it. Re-commitments are issued at the endpoint's deterministic setting (temperature 0, fixed seed where supported); otherwise the registered rule is k re-commitments with a majority verdict, and the gate's inferences assume deterministic J. Tool state cannot diverge between runs because the counterfactual plane is assembled from ledgered atoms, never by re-querying tools. A re-commitment that differs for a reason unrelated to the removed content is quarantined anyway: the conservative default, priced by the same false-quarantine rate.

5.2. Soundness and the Approximation Gap

Proposition 4 (Gate soundness, oracle case). Under Assumptions 1–2, deterministic J, and oracle plant removal ($\delta_1 \in \Delta_{\text{irrel}}(q)$): if the gate quarantines an admitted action a on the plant-removal plane, the executions on (q, d) and $(q, d \oplus \delta_1)$ are an I_5 witness pair at the level of policy-admissible decisions—$J_\Pi(x) = J(x)$ if $J(x)$ qualifies under admission and $\perp$ otherwise—with $\mathsf{dist} = 1 > \varepsilon$ under the discrete metric extended by $\mathsf{dist}(a, \perp) = 1$.

Proposition 5 (Approximation gap, deployable perturbations). Let the deployed gate approximate the plant-removal plane by an enforced set $E \subseteq \{\delta_2, \delta_3\}$, and over the registered probe distribution let *fp* be the rate at which an honest commitment changes under some $\delta \in E$ and *miss* the rate at which a laundered commitment survives every $\delta \in E$. For deterministic J the false-quarantine rate is at most *fp* and the missed-quarantine rate at most *miss*; each $\delta \in E$ is an exact recomputation inside the TCB and contributes no gap of its own.

Proofs: the artifact (faithful re-commitment; both quarantine disjuncts discharged at the policy-admissible level; qualification-preservation lemma and two event inclusions). *fp* and *miss* are measured in §6.3: δ_3 has *fp* 0.138 on actionable honest evaluations and *miss* 0.556 against the oracle comparator.

5.3. Relation to Counterfactual Defenses

Two concurrent efforts bound the same phenomenon from opposite sides: provenance-sensitivity auditing measures latent dependence offline and is disclaimed as a guardrail [1]; causal tool filtering minimizes which tools are visible [2]. Rationalization operates where filtering cannot help (the correct tool, legitimately cited), and the gate acts where offline diagnosis cannot (between admission and dispatch). Counterfactual re-execution defenses perturb a different plane: MELON masks the user prompt [13], AgentSentry purifies the untrusted context [14], CausalArmor ablates untrusted spans [15]; each asks whether the action depends on content it should ignore, whereas the gate asks whether the action stands on what it cites and quarantines irreversible classes before dispatch. Intervening at the (action, args) decision point rather than in token or latent space [3] is what makes the audit cheap enough to run per action.

Proposition 6 (Composition with sink authorization). For $P_{\text{DEI}} = I_1 \wedge I_2 \wedge I_3 \wedge I_4$ and any EM-enforceable safety property P_{sink} over the extended alphabet (a sink-authorization discipline [5]), the conjunction $P_{\text{DEI}}{\uparrow} \wedge P_{\text{sink}}$ is EM-enforceable by a product truncation automaton: every executed protected action is both evidentially sound and manifest-authorized, and the conjunction strictly refines each conjunct.

Proof: the artifact (padding preserves safety; product truncation automaton; commutativity; strictness witnesses in both directions). The practical reading is that DEI gates what may count as a reason and CXI-class systems gate what a proposal may do; neither subsumes the other.

6. Evaluation

Five questions. RQ1: does enforcing the safety fragment change attack outcomes on a live model that follows the injection, and how does it compare with simpler checks and an existing defense? RQ2: does the gate add security where admission is provably blind? RQ3: what does enforcement cost in benign utility, at the power needed to resolve it? RQ4: do the answers hold across models, suites, and attack families? RQ5: what does it cost in latency? Every number flows from the episode logs through a recompute pipeline that emits the macros used in the text.

6.1. Setup and Registered Design

Subjects, suites, attacks. The primary subject is a frontier live endpoint, gemini-2.5-flash, on the AgentDojo workspace suite (email, calendar, cloud drive; benchmark v1.2.x, package agentdojo 0.1.35) under the canonical important_instructions attack family, through a checkpointed harness derived from the benchmark's own run_task_with_pipeline path. The pilot

grid ran at the endpoint's default sampling temperature (the pilot harness did not set one); the registered harness sends temperature 0.7 and the derived seed explicitly on every request. A dated proprietary snapshot (gpt-4o-2024-05-13) is the positive calibration control (5/5 at t=0.0, 4/5 at t=0.7 on the known-susceptible workspace/user_task_0 injections), ruling out an inert harness. A 0.5B open-weight driver (Qwen2.5-0.5B-Instruct, greedy) developed the harness and the rationalization probes of §6.3; it completes no workspace task with or without enforcement (0/16), so no utility or attack rate is claimed from it. RQ4 adds a second frontier endpoint, a post-fix re-run of the open-weight qwen3 family, and the benchmark's tool_knowledge attack family, the first stage of the cascade of [50]; all three replications completed. AgentDyn adds the completed dynamic-suite replication reported in §6.5; suite-generality remains bounded by its task and model coverage [51].

Deployed policy objects. λ is as in Definition 1: the user turn and the system prompt TRUSTED, every tool result MEDIUM, model text no evidence at all. Support of an action is the set of atoms naming one of its argument values plus the user atoms that express its class (the registered directive lexicon); $S = \sum w/(1 + \sum w)$ over qualifying support with $\tau_c = 0.45$ at $\ell_{\text{req}} = \text{TRUSTED}$ for all protected classes, fixed before any attack run. This S satisfies A1–A3 of Definition 7 (sum over attested atoms at or above ℓ_{req}; $x/(1 + x)$ increasing with $w \geq 0$; weights fixed in the TCB from the recorded prefix). π authorizes no post-ingress raise. β binds targets that leave the trust boundary (recipients, payees, users, channels, URLs, amounts) at TRUSTED and internal handles that only tools can enumerate (file, e-mail, and event ids; hotel and vendor names) at MEDIUM; free text and dates are unbound. $D = \{\text{delete}, \text{share}, \text{send}, \text{book}\}$. Two-root corroboration is not deployable on a single first-party channel, so policy tightness is bounded by the attestation topology. The offline sweep gives a flat region at $\tau_c \in \{0.30, 0.45, 0.50\}$: 39/184 security and 22/51 benign first protected decisions admit. At $\tau_c = 0.60$ and 0.67, both rates fall to zero, placing the cliff between 0.50 and 0.60. Requiring every bound slot to be TRUSTED reduces security admissions to 19/ 184 but leaves benign admissions unchanged at 22/51.

Development versus test. Every case any development run touched is excluded from the held-out pools and reported as development evidence: the four rationalization probes, the five MCP authorization patterns of §4.4, the 22-task small-model workload on which τ_c was fixed, the post-fix open-weight run, and the 40-case/24-case pilot grid of §6.2, whose defended arms ran a protected-tool filter: the pilot harness never wired support extraction, so every state-changing call was dropped regardless of evidence, and the one target outside the filter's list (calendar-event creation) accounts for all four defended-arm breaches. The held-out case sets (150 security cases and the 56 benign tasks no development run touched, across the four suites, drawn by a seeded procedure), the 20 choice-laundering probes with their honest controls, the policy objects, the baseline and gate configurations, and the utility minimal important difference are deposited hash-sealed and publicly timestamped before the RQ1–RQ4 runs: registration v2 (archive SHA-256 ee88c72b…c68; manifest cbddbd33…f0; OpenTimestamps block 974214, 2026-11-02T05:06:44Z; the deleted original DOI is being re-registered at final upload as DOI https://doi.org/10.5281/zenodo.24075933) and the replacement-2 testbed v4 (DOI https://doi.org/10.5281/zenodo.23914405; archive SHA-256 d97f1369…f3b; manifest c67de998…071; OpenTimestamps block 974389, 2026-11-03T13:03:48Z). The campaign hotfix is DOI https://doi.org/10.5281/zenodo.23917748, manifest dfc691f8…13b, OpenTimestamps block 974430, 2026-11-03T19:47:14Z. An intermediate replacement-1 implementation and its results were excluded because they lacked independent restarts and MCP transport. The τ_c workload touched only benign traces; security cases on those workspace user tasks remain in the pool and form a registered descriptive stratum beside the τ_c sweep, rather than being silently presented as untouched.

Baselines. Three baselines run under the identical harness and case sets: B1, argument–instruction matching, every target named in a protected call must appear in the user's instruction (the simpler check a decision-layer defense must beat); B2, a tool-filter policy in the CaMeL lineage [4], [16] (no tool-sourced value may reach a protected sink unless the user turn names it); B3, I_4 alone. Every arm's verdict is logged on every protected call as a shadow; all seven arms completed the registered campaign.

Statistics. Wilson 95 % intervals on every rate; Fisher's exact test on the one pre-registered primary comparison per grid, with risk difference (RD) and risk ratio (RR) and their intervals beside every p-value; the rule-of-three bound beside any zero-event claim; secondary comparisons Holm-corrected. Five harness defects (among them the unset temperature) are documented in the artifact; degenerate episodes (null model output) are reported and excluded only in a sensitivity analysis. The reference semantics' own coverage check (10/10 classes, 3/3 honest baselines) is Appendix D.

6.2. RQ1: Security on a Live Model

The subject is susceptible. gemini-2.5-flash follows the canonical injection in 18/40 no-defense pilot episodes (ASR 0.45, Wilson 95 % CI $[0.307, 0.602]$); a disjoint 30-case sweep corroborates this ($23/60 = 0.383$ $[0.271, 0.510]$). The pilot covers three workspace user tasks, a scope the held-out set removes.

A protected-tool filter already collapses this family (pilot). On this subject (40 independent cases

TABLE 4. RQ1 and RQ3. Pilot (development evidence): gemini-2.5-flash, workspace, n=40 cases per security arm, n=24 per benign arm; the defended arms are two replicates of a protected-tool filter. Registered campaign: held-out cases, seven arms.

Arm	ASR	95% CI	Filtered
Pilot grid			
No defense	0.45 (18/40)	[0.307, 0.602]	0
Protected-tool filter, replicate 1	0.05 (2/40)	[0.014, 0.165]	47
Protected-tool filter, replicate 2	0.05 (2/40)	[0.014, 0.165]	48
Benign (n=24/arm)	no defense 20/24; filter 17/24 (§6.4)		
Registered campaign (150 security / 56 benign held-out cases)			
No defense	0.527 (79/150)	[0.447, 0.605]	0
Admission (I_1–I_4)	0.087 (13/150)	[0.051, 0.143]	114
Admission + gate	0.087 (13/150)	[0.051, 0.143]	110
B1 argument–instruction matching	0.087 (13/150)	[0.051, 0.143]	134
B2 tool filter	0.220 (33/150)	[0.161, 0.293]	124
B2strict filter	0.047 (7/150)	[0.023, 0.093]	146
B3 I_4 alone	0.167 (25/150)	[0.116, 0.234]	105

per arm), a filter that drops every call to a state-changing tool reduces measured attack success from 0.45 (18/40) to 0.05 (2/40, Wilson $[0.014, 0.165]$) in each of two replicates: Fisher exact p=0.0001, RD $= -0.400$ $[-0.568, -0.232]$, RR $= 0.111$ $[0.028, 0.448]$ (Table 4). The filter verifiably engages (47–48 protected calls dropped per arm); every surviving breach is the one template whose target, calendar-event creation, was not on its list, and excluding the four degenerate episodes changes nothing (2/37, p=6.4×10^{-5}). This is B2 in its strictest form, and it sets the bar: admission must match that security at lower benign cost. Security is measured across seven arms on the held-out set; the powered RQ3 crossover decides the comparative benign-cost bar. The registered rows show that admission reduces ASR from 0.527 to 0.087 (Fisher p=$2.9e-17$), but it does not beat B1: both are 0.087, with Holm p=$1.0e+00$. B2strict is lower still at 0.047. RQ1 is therefore a result about I_4 rather than evidence of full-DEI superiority; the gate does not change admission's ASR. The powered RQ3 contrasts, not the 56-task descriptive arm, decide comparative benign cost.

6.3. RQ2: What the Gate Adds

On the pilot grid the two defended arms are replicates of one filter (the pilot's gate re-commitment was not wired), so the pilot cannot show the gate's value, and the canonical injection family is caught at admission in any case. The gate is designed for the case admission cannot see (Cor. 2), and RQ2 is answered there.

Development probes: the counterexample, executed. Four probes pair a legitimate directive (TRUSTED, τ_c-sufficient) with a planted message naming an attacker-chosen target "with approval," on the small-model driver (existence evidence, not rates). In 8 runs the model followed the injection 4 times; twice it laundered the action under the genuine directive with τ_c-sufficient support, and I_1–I_3 admitted: Cor. 2 end to end. With the gate enabled (policy and perturbations frozen beforehand), both laundering admissions were quarantined on the oracle plane δ_1 (2 of 2); the honest-citation follow was blocked by I_3 before the gate; the interrupted follow, repaired to the user's target, re-committed identically under both perturbations. Of 3 gated benign commitments, 0 were falsely quarantined, at 1.375× model time and 52 μs p50 monitor latency. Cited-untrusted removal at $\ell_{\min}$ never triggered (0/26), which is why laundering passes I_1–I_3 at all.

What these probes do and do not show. They show that the gate closes the counterexample where I_1–I_3 are blind, not that it adds value over I_4: both laundered targets are absent from every trusted belief, so I_4 (and B1) blocks them at admission (Remark 3). The registered RQ2 design therefore uses choice-laundering probes, in which every bound argument is user-named and the plant is a false claim that selects among legitimate options: F1, delete whichever of two named files is older (the plant reverses the dates), placed inside the file listing the task reads (co-located) or inside a calendar entry it also reads (separate); F2, send to a named contact only if an event falls on a given day (the plant claims it moved); F3, write to whichever of two named contacts attends an event (the plant claims a substitution). Twenty probes with matched honest controls, four registered seeds each, validated so that each honest ground truth passes and each plant triggers the laundered action; I_4 (and B1) admit every laundered action by construction, so any quarantine is the gate's. The primary endpoints are pooled δ_3 catch on up to 40 separable-plant episodes and false quarantine on up to 60 actionable honest-control evaluations, with Wilson and probe-level cluster-bootstrap intervals and the actual denominators reported; non-action controls never enter a gate-flip denominator. Family/placement cells, observed δ_2, and oracle-δ_1 *fp/miss* per Prop. 5 are secondary; registered prediction: δ_3 catches the separate placement and F2 at no honest cost and cannot see co-located plants, which δ_2 flags only together with every honest commitment that needs the same atom. The outcome refutes zero honest cost: δ_3 catches 23/26 separable commitments (0.885, Wilson $[0.710, 0.960]$, probe-bootstrap $[0.704, 1.000]$) but falsely quarantines 4/29 controls (0.138, $[0.055, 0.306]$). Only 26 separable and 29 actionable evaluations arose; non-actions were not imputed. Co-located catch is 0.190, while δ_2 flags every co-located plant and 28/29 controls, the resolution limit. Against the oracle, δ_3 has TP 12, FN 15, FP 5, TN 2(miss 0.556, FP rate 0.714); δ_2 has TP 15, FN 12, FP 6, TN 1. The gate helps on some separable plants, but its atom-granularity and re-commitment failures are limits, not a general security claim.

6.4. RQ3: Utility Cost

Pilot. Under the protected-tool filter the benign-completion rate is 17/24 (0.71) versus 20/24 (0.83) with no defense, with failures 7/24 versus 4/24. The

12.5-point estimate is not resolvable at n=24 (Fisher p=0.49; paired discordant 3/0, exact McNemar p=0.25) and is structurally uninformative: 23 of the 24 benign tasks never call a protected tool, the one that does was blocked, and the two other extra failures occurred with nothing filtered. We claim neither that the cost is small nor that it is large; the registered study stratifies on protected-tool use.

Registered study. The utility comparison is re-run as a within-task crossover: each of the 97 benign tasks of the four suites runs under three registered seeds with no defense, admission plus gate, B1, and B2strict (291 pairs per contrast, registered random order), pairing the within-task variation an independent-arms design treats as noise. The sample size follows from a pre-registered minimal important difference of 10 points from the 83.3 % baseline (independent-arms floor 266 per arm at 80 % power; the paired test has power 0.83 at the independence bound and more under positive within-task correlation). Paired RD with a Newcombe interval for admission plus gate versus no defense is the headline; the paired contrasts against B2strict and B1, Holm-corrected, decide whether admission costs less than the filter; exact McNemar and a task-level cluster bootstrap is secondary; protected-tool-use and development-contact strata are descriptive; every failure is classified as defense-induced (blocked by a named invariant), model-induced, or degenerate. The registered crossover shows a material cost: DEI+gate changes completion by -0.268 relative to no defense (Newcombe $[-0.333, -0.199]$; task-cluster bootstrap $[-0.357, -0.182]$), not by a negligible amount. It nevertheless outperforms B2strict by 0.107 ($[0.057, 0.156]$; Holm p=0.000) and B1 by 0.055 ($[0.003, 0.106]$; Holm p=0.052), the latter borderline. DEI+gate failures split into 74 defense-induced, 115 model-induced, and 0 degenerate episodes; B2strict splits into 132 defense-induced and 88 model-induced. The practical claim is therefore a security–utility trade-off, not preserved utility.

6.5. RQ4: Generality

The pilot covers three workspace user tasks, one attack family, and one model (per-case susceptibility 0.18–1.00), so no cell extrapolates. Pre-fix open-weight archives (0/87 on qwen3-coder-30B, 0/33 on qwen3-32B) are negative observations, not resistance; a post-fix development run of qwen3.6-35B followed the injection in 18/60 no-defense episodes across the four suites, so RQ4 re-runs that family under the registration. On gpt-4.1-mini, ASR is 0.480 with no defense, 0.107 with DEI, 0.060 with DEI+gate, and 0.087 with B1. On tool_knowledge, the corresponding rates are 0.540, 0.100, and 0.080. All 150-security-case rows have Wilson intervals and zero voided outcomes; the benign rows are secondary and are not promoted to primary evidence. The open-weight row is completed but uninformative: qwen3:8b follows only 1/60 no-defense injections (DEI 0/60), so it does not establish susceptibility or robustness. AgentDyn replicates on 1,120 episodes with 30 voided attempts excluded: ASR falls from 0.470 to 0.237 (RD -0.232 $[-0.286, -0.178]$, Fisher $p = 4.8e-16$), with suite-generality stated only for the dynamic benchmark.

6.6. RQ5: Latency

Admission costs are those of Table 3: the replacement A2 gateway adds 2.7–10.1 % p50 overhead over A0 at 5–100 req/s, with mean p50 5.02 ms and mean p95 7.61 ms at 25 req/s. The gate adds one re-commitment per enforced perturbation on irreversible classes (1.375× benign model time in the development probes).

6.7. Threats to Validity

Construct and internal. DEI constrains evidential structure, not truth, and within-TCB compromise is excluded by the threat model. The pilot's defended arms ran a protected-tool filter at an unset temperature (§6.1); the registered harness fixes both, and B3 isolates I_4's contribution. Development probes were built against the gateway's own threat model; held-out sets are registered and timestamped before the runs.

External and statistical. CZTA is one instantiation on one deployment; efficacy is per model, suite, and attack family, and RQ4 bounds the generality claim. The pilot's security comparison has power ≈ 0.99 for its effect; its utility arm is underpowered; its defended arms are one filter, so RQ2 is answered where admission is provably blind.

Adaptive adversaries. I_5's perturbation class is registered but finite; an attacker outside it, or a model that re-commits identically with and without the plant, is the counterexample §3.5 invites.

7. Related Work

Defenses. Indirect prompt injection was identified early as the structural risk of tool-using LLM systems [19]; in-band defenses (detection classifiers, prompt hygiene, instruction-hierarchy fine-tuning) have been broken by adaptive evaluation [20]. Out of band, CaMeL separates control flow from data flow and attaches capability metadata to values [4]; its successors (FIDES, Progent, RTBAS, Conseca, FORGE) are systematized in [16] as Biba integrity plus reference monitoring plus least privilege, with two open problems that shape this paper: validation almost exclusively on static benchmarks, and an under-specified trusted base for provenance assignment. Guardrail agents [6], ARGUS's influence provenance graph [7], provenance surveys [42], and machine-checked Biba for agent memory [10] are CZTA's instantiation peers; none states the property it approximates as a definition. Counterfactual and causal defenses [1]–[3], [13]–[15] are compared with the gate in §5.

Integrity framings. Intent-to-execution integrity [17] gives judgment integrity the invariance form adopted here as I_5; agent epistemic integrity [18] is an architectural constraint secured by a model–harness contract; context-to-execution integrity [5] binds field, effect, and invocation authority to one manifest at the sink; a concurrent theory characterizes what a deterministic pre-execution gate can enforce [8]; semantic laundering [9] states the architectural version of the rationalization gap.

Benchmarks, protocol surface, foundations. AgentDojo established live-environment evaluation of injection [21], with documented scoring defects [50] and realism gaps [51]; sibling suites are consolidated in [22]. MCP standardized tool connectivity and authorization [23]; taxonomies rank tool poisoning among its principal risks [24], [25]; zero-trust MCP gateways decide who may call what [11], [12], not what may count as a reason; frontier-lab guidance treats internal agents as compromised insiders [43]. DEI applies classical machinery to a new object: integrity models [26]–[28], noninterference [29], decentralized IFC [30], provenance [31], evidence theory [32], [33], and the runtime-verification canon [34]–[41]. "Epistemic security" also names a societal program [44]; ours is the technical sense.

8. Conclusion

Defenses for agentic systems constrain what an agent may do. We have stated what it means for an agent's reasons to be trustworthy, proved which part a monitor can enforce and which it can only audit, and measured the resulting trade-offs. I_4 is the admission result: bound arguments must have cited evidential standing. The counterfactual gate is valuable on separable plants, but its misses, false quarantines, and utility costs are limits, not evidence of a general counterfactual defense.

Ethics considerations

This work evaluates defenses on a testbed the authors control; no production or third-party systems were probed, no third-party vulnerabilities were identified, and no responsible disclosure was required. The taxonomy of §2.1 derives from a registered literature review whose expert annotators coded publicly available bibliographic metadata against a registered codebook; no personal data about the annotators was collected, and only their per-record codes are released under role labels. No human-subjects study was conducted for this paper; a registered threshold-adjudication pilot described in the artifact awaits an ethics determination and contributes no result here. The attack content used in the evaluation is the public AgentDojo injection corpus and constructed probes against a local testbed; it confers no capability beyond what the benchmark already publishes.

LLM usage considerations

The authors used large language model assistants for editorial and preparatory support (phrasing, formatting, LaTeX, and consistency-checking of reported figures against source data), for assistance drafting definitions and proofs from author-specified theorem statements, and for developing the artifact's Lean 4 mechanization scripts. Every definition, theorem statement, and proof was reviewed and verified by the authors; every Lean script is checked by the Lean 4 kernel, which re-verifies each inference step independently of its provenance; every reported number is recomputed from episode logs by the pipeline in the artifact. The motivation for this use is editorial throughput and the mechanized re-verification it made affordable. The research questions, the framework, the system, and all contributions are the authors' own, and the authors take full responsibility for all material herein.

Availability

The anonymized artifact [49] contains the czta_ref reference monitor (hash-pinned at /env); the executable reference semantics and property tests (Appendix D); the Lean 4 mechanization (DeiSafety.lean) of the safety core, re-checkable by check_formal.sh; the scoping-review protocol, ledger, codebooks, instruments, per-record hashes, and amendment log; the measurement harness and per-run outputs behind Table 3; the authorization-layer harness and verdict logs; the live-model campaign behind Table 4 (the seed-audited harness g2_extended.py, both episode logs, the full console log, the recompute-and-macro pipeline, and a verification manifest whose SHA-256 hashes and recomputed figures match every number reported here); and the registered protocols, deposits, and timestamps for the completed runs of §6.

References

[1] J. Liao, "Auditing provenance sensitivity in LLM agent action selection," 2026, arXiv:2607.20827.

[2] R. S. Babu and R. Shukla, "GIST-CMTF: Goal-state inference for causal minimal tool filtering in LLM agents," 2026, arXiv:2606.16813.

[3] E. Pona, M. Kazemi, Y. Du, D. Watson, and N. Paoletti, "Abstract counterfactuals for language model agents," in Advances in Neural Information Processing Systems (NeurIPS), 2025; also arXiv:2506.02946.

[4] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, et al., "Defeating prompt injections by design," 2025, arXiv:2503.18813; IEEE SaTML 2026.

[5] I. Santos-Grueiro, "Context-to-execution integrity for LLM agents," 2026, arXiv:2607.06000.

[6] Z. Xiang et al., "GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning," 2024, arXiv:2406.09187; ICML 2025.

[7] S. Weng, Y. Feng, J. Zhang, X. Xie, J. Yu, and J. Liu, "ARGUS: Defending LLM agents against context-aware prompt injection," 2026, arXiv:2605.03378.

[8] S. Ray, "What can be enforced? A theory of certified runtime safety for tool-using agents," 2026, arXiv:2607.22868.

[9] O. Romanchuk and R. Bondar, "Semantic laundering in AI agent architectures: Why tool boundaries do not confer epistemic warrant," 2026, arXiv:2601.08333.

[10] Y. Louck, "Securing LLM-agent long-term memory against poisoning: Non-malleable, origin-bound authority with machine-checked guarantees," 2026, arXiv:2606.24322.

[11] S. Mishra, "ZT-MCP: A zero-trust security architecture for MCP-connected AI agents," 2026, ResearchGate preprint, doi:10.13140/RG.2.2.36042.53445.

[12] S. V. Kodathala, "aiAuthZ: Off-host, identity-bound authorization for AI agents," 2026, arXiv:2607.05518.

[13] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "MELON: Provable defense against indirect prompt injection attacks in AI agents," in Proc. ICML, 2025; also arXiv:2502.05174.

[14] T. Zhang, Y. Xu, J. Wang, et al., "AgentSentry: Mitigating indirect prompt injection in LLM agents via temporal causal diagnostics and context purification," 2026, arXiv:2602.22724.

[15] M. Kim, M. Parmar, P. Wallis, et al., "CausalArmor: Efficient indirect prompt injection guardrails via causal attribution," 2026, arXiv:2602.07918.

[16] LaunchSafe Research, "Adaptive evaluation of out-of-band defenses against prompt injection in LLM agents," 2026, arXiv:2606.26479.

[17] W. Qu, M. Xu, P. Wang, S. Zhai, J. Zhang, and D. Song, "Securing LLM agents need intent-to-execution integrity," 2026, arXiv:2605.16976.

[18] Z. Shen, "Neither layer alone: Epistemic integrity requires hierarchical joint design for long-running AI agents," 2026, arXiv:2606.04017, posted 1 June 2026.

[19] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023.

[20] S. Abdelnabi and E. Bagdasarian, "AI agents may always fall for prompt injections," 2026, arXiv:2605.17634.

[21] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in NeurIPS Datasets and Benchmarks, 2024.

[22] M. Q. Li, B. C. M. Fung, B. Li, H. Ismail, and F. Iqbal, "Taxonomy and consistency analysis of safety benchmarks for AI agents," 2026, arXiv:2605.16282.

[23] Model Context Protocol contributors, "Model context protocol specification," Specification revision 2025-11-25 (authorization framework), 2025.

[24] OWASP Foundation, "OWASP MCP top 10 (beta)," 2026; Tool Poisoning listed (MCP03).

[25] M. Rostamzadeh, S. Narula, N. Birhan, M. Ghasemigol, and D. Takabi, "MCP-DPT: A defense-placement taxonomy and coverage analysis for MCP security," 2026, arXiv:2604.07551.

[26] K. J. Biba, "Integrity considerations for secure computer systems," MITRE Corporation, Tech. Rep. ESD-TR-76-372, 1977.

[27] D. E. Denning, "A lattice model of secure information flow," Communications of the ACM, vol. 19, no. 5, pp. 236–243, 1976.

[28] D. D. Clark and D. R. Wilson, "A comparison of commercial and military computer security policies," in IEEE Symposium on Security and Privacy, 1987.

[29] J. A. Goguen and J. Meseguer, "Security policies and security models," in IEEE Symposium on Security and Privacy, 1982.

[30] A. C. Myers and B. Liskov, "A decentralized model for information flow control," in ACM Symposium on Operating Systems Principles (SOSP), 1997.

[31] W3C, "PROV-DM: The PROV data model," W3C Recommendation, 30 April 2013, 2013.

[32] G. Shafer, A Mathematical Theory of Evidence. Princeton University Press, 1976.

[33] P. M. Dung, "On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games," Artificial Intelligence, vol. 77, no. 2, pp. 321–357, 1995.

[34] B. Alpern and F. B. Schneider, "Recognizing safety and liveness," Distributed Computing, vol. 2, no. 3, pp. 117–126, 1987.

[35] F. B. Schneider, "Enforceable security policies," ACM Transactions on Information and System Security, vol. 3, no. 1, pp. 30–50, 2000.

[36] A. Bauer, M. Leucker, and C. Schallhart, "Runtime verification for LTL and TLTL," ACM Transactions on Software Engineering and Methodology, vol. 20, no. 4, 2011.

[37] A. Pnueli and A. Zaks, "PSL model checking and run-time verification via testers," in International Symposium on Formal Methods (FM), 2006.

[38] F. Chen and G. Roşu, "MOP: An efficient and generic runtime verification framework," in ACM OOPSLA, 2007.

[39] M. R. Clarkson and F. B. Schneider, "Hyperproperties," Journal of Computer Security, vol. 18, no. 6, pp. 1157–1210, 2010.

[40] B. Finkbeiner, C. Hahn, M. Stenger, and L. Tentrup, "Monitoring hyperproperties," Formal Methods in System Design, vol. 54, pp. 336–363, 2019.

[41] S. Stucki, C. Sánchez, G. Schneider, and B. Bonakdarpour, "Gray-box monitoring of hyperproperties," in International Symposium on Formal Methods (FM), 2019.

[42] Y. Wang et al., "From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents," 2026, arXiv:2606.04990.

[43] M. Phuong, E. Jenner, L. Simon, L. Ho, R. Shah, S. Farquhar, and S. Coull, "AI control roadmap / three layers of agent security," Google DeepMind report and blog, 18 June 2026.

[44] E. Seger, S. Avin, G. Pearson, M. Briers, S. Ó hÉigeartaigh, and H. Bacon, "Tackling threats to informed decision-making in democratic societies: Promoting epistemic security in a technologically-advanced world," The Alan Turing Institute, Tech. Rep., 2020.

[45] A. C. Tricco et al., "PRISMA extension for scoping reviews (PRISMA-ScR): Checklist and explanation," Annals of Internal Medicine, vol. 169, no. 7, pp. 467–473, 2018.

[46] K. L. Gwet, "Computing inter-rater reliability and its variance in the presence of high agreement," British Journal of Mathematical and Statistical Psychology, vol. 61, no. 1, pp. 29–48, 2008.

[47] J. Cohen, "A coefficient of agreement for nominal scales," Educational and Psychological Measurement, vol. 20, no. 1, pp. 37–46, 1960.

[48] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927.

[49] Anonymous, "Epistemic security artifact: reference monitor, reference semantics, Lean 4 mechanization, review ledger, and campaign logs," anonymized repository, https://anonymous.4open.science/r/epistemic-security-artifact-AF35/.

[50] R. Bhagwatkar, K. Kasa, A. Puri, G. Huang, I. Rish, G. W. Taylor, K. Dj Dvijotham, and A. Lacoste, "Indirect prompt injections: Are firewalls all you need, or stronger benchmarks?" 2025, arXiv:2510.05244.

[51] H. Li, R. Wen, S. Shi, N. Zhang, Y. Vorobeychik, and C. Xiao, "AgentDyn: Are your agent security defenses deployable in real-world dynamic environments?" 2026, arXiv:2602.03117.

Appendix A. Proof Details

(a) Setup and violation predicates. Fix the event alphabet $\Sigma = \{\mathsf{ing}(e), \mathsf{upd}(b, X), \mathsf{end}(b, \ell, \rho), \mathsf{act}(a)\}$ over the runtime object universes of §3. A finite prefix $\sigma[..k]$ determines a monitor state: the observed evidence and belief sets, the label map $\ell(\cdot)$, the dependency graph P, for each observed belief b its recorded value set $\mathsf{vals}(b)$, and a memoized root certificate $\mathsf{cert}(b) \in \{\mathrm{ok}, \mathrm{bad}\}$ computed when b is created (ok iff every maximal derivation path from b terminates in evidence e with $\mathsf{att}(e)$ and $\ell(e) \sqsupseteq \ell_{\min}$; certificates of parents are reused, giving the $O(1)$ amortized update of Remark 1). All four conditions below are decidable from the prefix state at the event that would violate them.

(b) I_1–I_4 are safety properties. By Alpern–Schneider [34], a property is safety iff every violating infinite behavior has a finite bad prefix that no extension can repair. Define the per-event violation predicates: $\mathsf{viol}_1(\sigma[..k])$ iff $\epsilon_k = \mathsf{act}(a)$ with $c(a) \in C_{\mathrm{prot}}$ and some $b \in \mathsf{support}(a)$ has $\mathsf{cert}(b) = \mathrm{bad}$ (or an ill-founded closure), or fails the action-time freshness recheck $\neg\mathrm{WindowValid}(b)$ (a terminal root's attestation validity window has lapsed at the time of ϵ_k, though its creation-time certificate may be ok; Def. 2 evaluates att against the current time); $\mathsf{viol}_2(\sigma[..k])$ iff ϵ_k raises some label other than by an authorized end event, or $\epsilon_k = \mathsf{upd}(b', X)$ assigns $\ell(b') \not\sqsubseteq \bigsqcap_{x \in X} \ell(x)$; $\mathsf{viol}_3(\sigma[..k])$ iff $\epsilon_k = \mathsf{act}(a)$ protected and $S(\mathsf{support}(a), \ell_{\mathrm{req}}(c(a))) < \tau_{c(a)}$; $\mathsf{viol}_4(\sigma[..k])$ iff $\epsilon_k = \mathsf{act}(a)$ protected and some bound slot $s \in \beta(c(a))$ with $\mathsf{args}(a)(s)$ defined has no $b \in \mathsf{support}(a)$ with $\mathsf{args}(a)(s) \in \mathsf{vals}(b)$ and $\ell(b) \sqsupseteq \ell_\beta(c(a), s)$. Each predicate depends only on $\sigma[..k]$. A trace violates I_j iff some prefix satisfies viol_j; the offending event has occurred and occurs in every extension, so the prefix is bad and irremediable: I_1–I_4 are safety. Monitorability follows: the LTL_3 semantics [36], [37] assigns verdict $\bot$ exactly at the first prefix satisfying viol_j. Quantification over beliefs and actions is resolved by parametric monitoring [38]: the object universe at any prefix is the finite set of objects observed so far, and each instantiated monitor is the two-state automaton ok $\to$ bad driven by viol_j.

(c) Enforceability by truncation. Schneider's execution monitors enforce exactly policies for which a monitor mediating each event can halt the target before an unacceptable step commits [35]. The gateway mediates every Σ-event (complete-mediation assumption of §2.2); on a candidate event ϵ it evaluates viol_j against the current state and suppresses ϵ iff some predicate fires. Two facts make this a sound truncation automaton. (i) Blocking preserves legality: I_1–I_4 universally quantify over events present in the trace, so deleting a candidate event can never create a violation; the truncated trace extends the previously legal prefix by nothing and remains legal. (ii) Blocking loses nothing legal: suppression occurs only when the event itself is the bad event, so every I-satisfying continuation is preserved. Hence $I_1 \wedge I_2 \wedge I_3 \wedge I_4$ is EM-enforceable, and the enforcement cost is the per-event predicate evaluation measured in Table 3 and Appendix D; the viol_4 test adds set membership on recorded value sets and changes neither fact.

(d) I_5 is 2-safety and not single-trace monitorable. I_5 constrains pairs of executions: it is the hyperproperty $H = \{T : \forall \sigma, \sigma' \in T$ agreeing on q and trusted state, with $d' = d \oplus \delta$, $\delta \in \Delta_{\mathrm{irrel}}(q) : \mathsf{dist}(J_\sigma, J_{\sigma'}) \le \varepsilon\}$, refutable by exhibiting two finite traces: 2-safety in the sense of Clarkson–Schneider [39]. It is not a trace property: membership of an individual trace is undefined. For non-monitorability, consider any black-box monitor M mapping single observed prefixes to verdicts, and two systems S^+ (satisfying I_5) and S^- (violating it on some δ-counterfactual) constructed to produce the same behavior on the observed input (q, d); the violation of S^- is witnessed only by the unobserved counterfactual run on $(q, d \oplus \delta)$. M receives identical input under both systems, so any verdict it issues is unsound for one of them; no sound single-trace black-box monitor can refute I_5. Runtime checking therefore requires stored comparison executions or a model of the system (gray-box monitoring [40], [41]), and we realize the check as the offline pre-registered differential harness of §3.5 and, before dispatch, as the gate of §5.

Appendix B. Charting-Class to Invariant Crosswalk

The full crosswalk table, with per-class mechanisms, refutation signals, and corpus shares, is included in the anonymized artifact rather than printed here.

Appendix C. Mechanism Predicates and Proofs of Theorem 1 and Corollary 2

Definition 11 (Mechanism realization). For each charting class K with a machine-fired primary $j(K) \in \{1, 2, 3\}$ (the nine machine-fired rows of Appendix B: eight full classes plus the anchor-extension boundary case), the trace predicate $\mathrm{Mech}_K(\sigma)$ holds iff σ has a prefix $\sigma[..k]$ whose final event ϵ_k satisfies the row's

condition, stated in the vocabulary of Appendix A(a): hallucination (j=1), a protected $\mathsf{act}(a)$ with some $b \in \mathsf{support}(a)$ whose closure has a maximal derivation path terminating in no evidence atom (Def. 5(i)), so $\mathsf{cert}(b) =$ bad; provenance/identity tampering (j=1), a terminal root of the support closure that fails att (Def. 2); retrieval/knowledge poisoning (j=1), a terminal root with $\neg\mathsf{att}(e)$ or $\ell(e) \not\sqsupseteq \ell_{\min}$; drift/staleness ($j$=1, partial), a terminal root whose validity window has lapsed at action time, so $\neg$WindowValid(b); prompt injection (j=2), an $\mathsf{upd}(b', X, \ell_{\text{prop}})$ with $\ell_{\text{prop}} \not\sqsubseteq \sqcap_{x\in X}\ell(x)$ or an $\mathsf{end}(b, \ell', \rho)$ with $\ell' \sqsupset \ell(b)$ and ($\rho \notin \pi$ or $\neg$Authorized), the affected belief depending on untrusted-channel content; confidence/trust manipulation (j=2), an endorsement attempt citing no authorized rule; tool/agent hijacking (j=2, partial), a label-raising upd or end without authorized endorsement on a belief depending on tool-result content; misinformation (j=3), a protected $\mathsf{act}(a)$ with $S(\mathsf{support}(a), \ell_{\text{req}}(c(a))) < \tau_c$ once non-qualifying evidence is excluded by A1; retrieval boundary case (j=3), the same deficit arising from selection starving qualifying support from an intact substrate. The declared out-of-scope row and the control row carry no predicate.

Proof of Theorem 1. Let $\text{Mech}_K(\sigma)$ hold with witness prefix $\sigma[..k]$. The four j=1 clauses are exactly the bad-certificate or lapsed-window disjuncts of $viol_1$; the three j=2 clauses are label raises without an authorized endorsement, exactly $viol_2$; and the two j=3 clauses are the qualifying-score deficit $viol_3$, with A1 explaining which atoms contribute. Thus $viol_{j(K)}(\sigma[..k])$ holds. These predicates are prefix-persistent, so a first firing index $k^* \le k$ exists. Assumption 1 makes that event mediated, Assumption 2 makes the recorded prefix state trusted, and Algorithm 1 suppresses precisely the firing event; it therefore blocks at or before ϵ_k. Conversely, a control instance fires no predicate. Since the algorithm blocks only on a predicate and blocking loses nothing legal (Appendix A(c)), every control event is admitted. The 10/10 attack and 3/3 honest reference-suite outcomes (Appendix D) validate realization of the predicates; they are empirical checks, not premises of this trace-level proof. □

Proof of Corollary 2. (i) Existence. Take an honestly admitted class-c_0 action with support S_0: its roots are attested and fresh and $S(S_0, \ell_{\text{req}}(c_0)) \ge \tau_c$. The adversary places d_{inj} on an untrusted channel. The compromised model selects any $a^* \in A_\beta(S_0)$ from that injection but commits the genuine S_0, with no label raise or endorsement. At the action prefix, all certificates and windows are valid ($\neg viol_1$), labels satisfy the meet ($\neg viol_2$), the score passes ($\neg viol_3$), and every bound slot is named at the required label ($\neg viol_4$). A sound truncation monitor must admit this legal event although injection caused the selection. Because the certificates, score, and binding test read only genuine attestation, labels, weights, and recorded values, finer provenance granularity changes none of them.

(ii) Necessity. Suppose a single-trace detector D soundly blocks this strategy. Let δ_1 remove the plant. Construct an honest S^+ that commits a^* both with and without the plant, and a laundering S^- that emits the identical observed stream on (q, d) but commits $a^- \neq a^*$ after removal. Both observed streams satisfy I_1–I_4, so D must issue the same verdict for indistinguishable systems: blocking is unsound for S^+, admitting for S^-. This is the $S^\pm$ construction of Prop. 2; detection therefore requires a pair-witnessing differential harness or the decision-local gate, not a refined single-trace admission rule. □

Appendix D. Executable Reference Semantics and Evidence Maturity

The executable monitor implements Algorithm 1 event for event: lattice-meet label propagation, authorized endorsement, fresh root certificates, support scoring, and bound-slot checks precede every protected commit. Its constructed suites exercise each crosswalk predicate, the I_5 differential harness, and truncation legality. Table 5 reports these semantics checks separately from live-model efficacy.

TABLE 5. Executable semantics checks; counterfactual results are invariance followed by witnesses.

Property under test	Result
Predicted-invariant blocks; coverage counterexamples	10/10; 0
Honest admissions; admit/block and legality tests	3/3; 13/13
Injection-obeying decision function	71%; 6/21
Guarded decision function	100%; 0/21
Memoized cost span; unmemoized depth correlation	1.42×; 0.996
Development verdict/action invariance	52/60; 41/60

Across 12 tasks and five Δ_{irrel} families, reordering remained invariant (12/12), while inject-then-ignore was sharpest (7/12). These are development stability checks, not live security estimates. The 256× depth experiment supports Remark 1: memoized receipt checks remain nearly flat while unmemoized cost tracks depth.

Evidence maturity. Props. 1–2 and Cor. 2, with the Lean 4 safety mechanization, are proved; the replacement-2 I_4 bound-slot check is implemented. Table 5, the authorization cases, and RQ1–RQ5 are measured: 1,442 RQ1 episodes with zero voided and a DEI/B1 tie; 640 RQ2 probes plus 80 oracle episodes; 1,164 RQ3 episodes with 50 voided and one degenerate, giving 291 pairs per contrast; 27/27 RQ5 cells; 824 RQ4(a) and 618 RQ4(c) episodes; and 1,120 AgentDyn episodes with 30 voided attempts excluded. RQ4(b) completed 120 episodes but remains uninformative at 1/60 susceptibility. Development checks are not held-out effectiveness evidence.